Bài Tập về đồ thị

Những bài tập cơ bản nhất về dồ thị

Nguyễn Văn Phú - Hà Minh Đức - Nguyễn Tấn Đạt

THPT Chuyên ĐHV

Ngày 18 tháng 3 năm 2026

Bài toán 1 - Đếm thành phần liên thông

Đề bài

Cho một đồ thị vô hướng gồm n đỉnh và m cạnh. Hãy đếm số **thành phần liên thông**.

Bài toán 1 - Đếm thành phần liên thông

Đề bài

Cho một đồ thị vô hướng gồm n đỉnh và m cạnh. Hãy đếm số **thành phần liên thông**.

Gợi ý

Sử dụng DFS hoặc BFS để duyệt từng thành phần.

Bài toán 1 - Đếm thành phần liên thông

Đề bài

Cho một đồ thị vô hướng gồm n đỉnh và m cạnh. Hãy đếm số **thành phần liên thông**.

Gợi ý

Sử dụng DFS hoặc BFS để duyệt từng thành phần.

Link tham khảo:

Click vào đây để luyện tập thêm

Solution - Bài 1

Ý tưởng

- Duyệt qua từng đỉnh
- Nếu đỉnh chưa thăm $\rightarrow$ chạy DFS/BFS
- Mỗi lần chạy $\rightarrow$ tăng số thành phần lên 1

Solution - Bài 1

Ý tưởng

- Duyệt qua từng đỉnh
- Nếu đỉnh chưa thăm $\rightarrow$ chạy DFS/BFS
- Mỗi lần chạy $\rightarrow$ tăng số thành phần lên 1

Mã giả DFS

```
for u in V:
    if not visited[u]:
        DFS(u)
        count += 1
```

Bài toán 2 - Đường đi ngắn nhất

Đề bài

Cho đồ thị không trọng số. Tìm khoảng cách ngắn nhất từ đỉnh s đến tất cả các đỉnh còn lại.

Bài toán 2 - Đường đi ngắn nhất

Đề bài

Cho đồ thị không trọng số. Tìm khoảng cách ngắn nhất từ đỉnh s đến tất cả các đỉnh còn lại.

Gợi ý

Sử dụng BFS để đảm bảo đi theo từng lớp.

Bài toán 2 - Đường đi ngắn nhất

Đề bài

Cho đồ thị không trọng số. Tìm khoảng cách ngắn nhất từ đỉnh s đến tất cả các đỉnh còn lại.

Gợi ý

Sử dụng BFS để đảm bảo đi theo từng lớp.

Link tham khảo:

Click để thử sức

Solution - Bài 2

Ý tưởng

- Khởi tạo khoảng cách $dist[s] = 0$
- Dùng queue để BFS
- Mỗi lần đi qua cạnh $\rightarrow$ cập nhật khoảng cách

Mã giả BFS

```
vector<int> adj[N];
bool visite [N];
int dist[N];
void bfs(int s) {
    queue<int> q;
    q.push(s);
    visite [s] = true;
    dist[s] = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u]) {
            if (visite [v]) continue;
            visite [v] = true;
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
}
```

Bài toán 3 - Phát hiện chu trình

Đề bài

Cho đồ thị vô hướng. Kiểm tra xem đồ thị có chứa chu trình hay không.

Bài toán 3 - Phát hiện chu trình

Đề bài

Cho đồ thị vô hướng. Kiểm tra xem đồ thị có chứa chu trình hay không.

Gợi ý

Dùng DFS và lưu đỉnh cha.

Bài toán 3 - Phát hiện chu trình

Đề bài

Cho đồ thị vô hướng. Kiểm tra xem đồ thị có chứa chu trình hay không.

Gợi ý

Dùng DFS và lưu đỉnh cha.

Link luyện tập:

Cycle Detection - CSES

Solution - Bài 3

Ý tưởng

- Khi DFS từ u sang v
- Nếu v đã thăm và $v \neq parent[u] \rightarrow$ có chu trình

Solution - Bài 3

Ý tưởng

- Khi DFS từ u sang v
- Nếu v đã thăm và $v \neq parent[u] \rightarrow$ có chu trình

Mã giả DFS

```
DFS(u, parent):
    visited[u] = true
    for v in adj[u]:
        if not visited[v]:
            if DFS(v, u): return true
        else if v != parent:
            return true
    return false
```

Tổng kết luyện tập

Những dạng quan trọng

- DFS $\rightarrow$ thành phần liên thông, chu trình
- BFS $\rightarrow$ đường đi ngắn nhất
- Kết hợp $\rightarrow$ nhiều bài nâng cao

Tổng kết luyện tập

Những dạng quan trọng

- DFS $\rightarrow$ thành phần liên thông, chu trình
- BFS $\rightarrow$ đường đi ngắn nhất
- Kết hợp $\rightarrow$ nhiều bài nâng cao

Practice makes perfect!